

Arquitectura de Microservicios y Patrones de Diseño

Asignatura: DSY1106 - DESARROLLO FULLSTACK III

Caso Semestral: Innovatech Solutions – Plataforma Inteligente para la gestión integral de proyectos tecnológicos

Integrante: Richard Moreano

Fecha: Mayo 2026

1. Introducción

La transformación digital ha impulsado a las organizaciones a adoptar arquitecturas de software más flexibles, escalables y mantenibles. En este contexto, Innovatech Solutions enfrenta desafíos asociados a la gestión de múltiples proyectos, la asignación eficiente de recursos y el monitoreo del desempeño organizacional.

El presente informe tiene como objetivo diseñar una solución tecnológica basada en una arquitectura de microservicios, incorporando arquetipos y patrones arquitectónicos que permitan estructurar el sistema de manera eficiente. Asimismo, se integran mecanismos de seguridad mediante autenticación con JSON Web Token (JWT) y se aplican patrones de diseño tanto en el frontend como en el backend.

De acuerdo con Sam Newman (2021), la arquitectura de microservicios permite construir sistemas altamente escalables mediante la descomposición de funcionalidades en servicios independientes.

Contexto del problema

Innovatech Solutions es una empresa de desarrollo de software que cuenta con más de 120 colaboradores distribuidos en equipos multidisciplinarios. La organización enfrenta dificultades en la gestión simultánea de múltiples proyectos, lo que genera problemas de coordinación, asignación de recursos y monitoreo del desempeño.

Entre los principales desafíos se identifican:

- Falta de visibilidad en tiempo real del estado de los proyectos
- Ineficiencia en la asignación de recursos humanos
- Ausencia de indicadores clave de desempeño (KPI) centralizados

Estos problemas afectan directamente la toma de decisiones estratégicas y la eficiencia operativa.

Objetivo General

Diseñar una solución basada en arquitectura de microservicios que permita gestionar proyectos, recursos y métricas organizacionales de manera eficiente y escalable.

Objetivos Específicos

- Implementar una arquitectura desacoplada mediante microservicios
- Aplicar patrones de diseño que mejoren la mantenibilidad
- Integrar mecanismos de autenticación seguros mediante JWT

- Facilitar la comunicación mediante APIs REST

2. Propuesta de arquitectura

La solución propuesta se basa en una arquitectura de microservicios, donde cada componente del sistema opera de manera independiente.

Componentes principales:

- Frontend (React)
- Backend For Frontend (BFF)
- API Gateway
- Microservicios:
 - Gestión de Proyectos
 - Gestión de Recursos
 - Analítica
 - Autenticación (JWT)
- Bases de datos independientes

Flujo del sistema:

Usuario → Frontend → BFF → API Gateway → Microservicios → Base de Datos

Esta arquitectura permite escalabilidad, independencia de servicios y facilidad de mantenimiento.

3. Arquetipos utilizados

Los arquetipos permiten establecer una base estructural para el desarrollo del sistema.

Backend

Se utiliza Spring Boot Initializr, que permite generar una estructura base con:

- Configuración inicial del proyecto
- Dependencias (JPA, seguridad, REST)
- Organización en capas
- Spring Web (API REST)
- Spring Data JPA (persistencia)
- Spring Security (seguridad)
- Spring Boot DevTools (desarrollo)
- PostgreSQL Driver (base de datos)
- Lombok (reducción de código boilerplate)

- Spring Cloud Gateway (API Gateway)
- Resilience4j (Circuit Breaker)

Frontend

Se utiliza React con Vite, proporcionando:

- Estructura modular
- Componentes reutilizables
- Configuración de rutas y estado

Estos arquetipos aceleran el desarrollo y garantizan buenas prácticas.

4. Patrones arquitectónicos

Se implementan los siguientes patrones:

Microservicios

Permite dividir el sistema en servicios independientes.

API Gateway

Centraliza las solicitudes del cliente y gestiona la comunicación.

Database per Service

Cada microservicio posee su propia base de datos.

Circuit Breaker

Evita fallos en cascada entre servicios.

La selección de estos patrones responde a la necesidad de desacoplar los componentes del sistema, mejorar la escalabilidad y garantizar la resiliencia. En particular, el uso de microservicios permite gestionar de forma independiente los módulos de proyectos, recursos y analítica, mientras que el API Gateway centraliza el acceso del cliente y el Circuit Breaker protege ante fallos en servicios externos.

5. Patrones de diseño

Backend (Microservicios)

Repository Pattern: Implementado en las clases ProyectoRepository y RecursoRepository, las cuales extienden de JpaRepository. Esto permite abstraer completamente las consultas a la base de datos (ejemplo: findByEstado(String estado)), facilitando cambios futuros de tecnología de persistencia y pruebas unitarias.

```
public interface ProyectoRepository extends JpaRepository<Proyecto, Long> {  
    List<Proyecto> findByEstado(String estado);  
}
```

Factory Method: Implementado en ProyectoFactory y RecursoFactory. Estos factories centralizan la creación de entidades con valores por defecto (estado "PENDIENTE", fecha de inicio actual), evitando duplicación de código y mejorando la consistencia.

DTO Pattern: Se crearon clases específicas (ProyectoRequest, ProyectoResponse, RecursoRequest, RecursoResponse) para transferir datos entre capas sin exponer las entidades de dominio directamente.

BFF

Circuit Breaker Pattern: Implementado utilizando Resilience4j con la anotación @CircuitBreaker en los métodos del BffController, junto con métodos de fallback que devuelven respuestas controladas cuando los microservicios no responden.

Frontend

Component-Based Architecture (CBA): Desarrollado el componente ProjectCard.jsx, el cual se preparó como módulo NPM reutilizable, permitiendo su consumo en otros proyectos.

Observer Pattern: Implementado mediante los hooks useState y useEffect para reaccionar a cambios de estado y actualizar automáticamente la interfaz al consumir datos del BFF.

```
const [proyectos, setProyectos] = useState([]);

useEffect(() => {
  fetchKPI();
}, [proyectos]);

const agregarProyecto = () => {
  setProyectos([...proyectos, nuevoProyecto]);
};
```

Estos patrones mejoran la modularidad y mantenibilidad del sistema, permitiendo construir una solución desacoplada, escalable y resiliente, alineada con la necesidad de Innovatech Solutions de monitoreo en tiempo real y asignación dinámica de recursos. Como señala Villamizar et al. (2015), la arquitectura monolítica obliga a escalar toda la aplicación aunque solo un servicio esté saturado; los microservicios resuelven este problema permitiendo escalado granular.

6. Integración del sistema (BFF)

Para facilitar la comunicación entre frontend y backend se implementa un Backend For Frontend (BFF).

Funciones del BFF:

- Adaptar respuestas de microservicios
- Reducir llamadas innecesarias
- Simplificar la lógica del frontend

Flujo de integración:

Frontend → BFF → API Gateway → Microservicios

Esto permite mejorar el rendimiento y reducir la complejidad.

7. Autenticación y seguridad

Se implementa autenticación basada en JWT (JSON Web Token).

Funcionamiento:

1. El usuario inicia sesión

2. El servicio de autenticación genera un token
3. El cliente envía el token en cada solicitud
4. El API Gateway valida el token

```
String token = Jwts.builder()  
    .setSubject(username)  
    .setIssuedAt(new Date())  
    .setExpiration(new Date(System.currentTimeMillis() + 3600000)) // 60 minutos  
    .signWith(secretKey)  
    .compact();
```

Beneficios:

- Autenticación sin estado (stateless)
- Seguridad en la comunicación
- Control de acceso basado en roles

El uso de JWT es especialmente adecuado en arquitecturas de microservicios debido a su naturaleza stateless, lo que facilita la escalabilidad del sistema.

8. Justificación de herramientas y estrategias

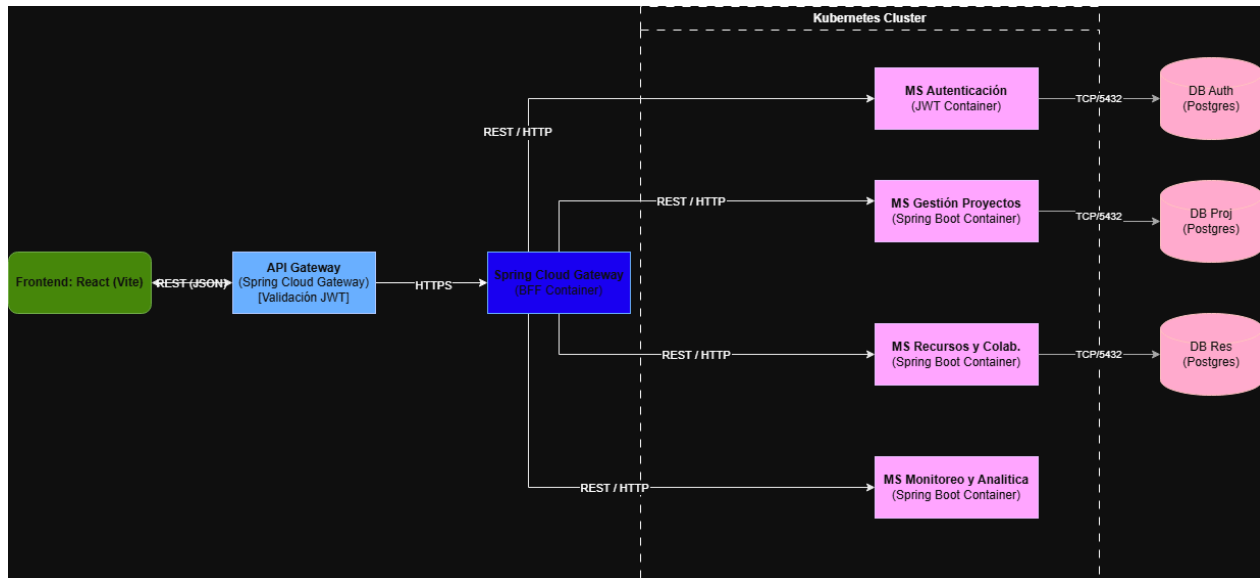
Se proponen las siguientes herramientas y estrategias:

- Backend: Spring Boot + Spring Cloud (arquetipos Maven personalizados)
- API Gateway: Spring Cloud Gateway
- Comunicación: REST + JSON
- Persistencia: JPA + PostgreSQL (Database-per-Service)
- Resiliencia: Resilience4j (Circuit Breaker)
- Orquestación: Docker + Kubernetes (o Docker Compose para desarrollo)
- Monitoreo: Prometheus + Grafana

Aporte a la eficiencia técnica y operativa (Villamizar et al., 2015):

- Permite despliegues independientes y continuous delivery.
- Reduce costos de infraestructura en aproximadamente 17% al asignar instancias más pequeñas y específicas a cada microservicio.
- Facilita el crecimiento de equipos: cada microservicio puede ser desarrollado por un equipo pequeño y autónomo.

9. Diagrama de la arquitectura de microservicios propuesta



10. Consideraciones de seguridad, privacidad y sostenibilidad

- Seguridad:
 - Autenticación y autorización con JWT + OAuth2 en el API Gateway.
 - Cada microservicio valida el token.
 - Comunicación interna y externa mediante HTTPS.
 - Circuit Breaker evita ataques de denegación de servicio en cascada.
- Privacidad:
 - Cumplimiento de la Ley 19.628 (Chile) y principios GDPR.
 - Datos personales encriptados en reposo (AES-256) y en tránsito.
 - Control de acceso basado en roles (RBAC).
- Sostenibilidad:
 - Uso de contenedores Docker + Kubernetes permite escalar solo los servicios necesarios, optimizando recursos y reduciendo consumo energético.
 - Monitoreo continuo con Prometheus + Grafana.
 - Selección granular de instancias en la nube reduce el impacto ambiental.

11. Implementación (Parcial 2)

Se ha procedido a la implementación real de la arquitectura propuesta en el Parcial 1, materializando los siguientes componentes:

- **Frontend:** React + Vite con componente ProjectCard listo como NPM Module.
- **BFF:** Backend For Frontend con orquestación y Circuit Breaker.

- **Microservicios:** Gestión de Proyectos y Gestión de Recursos (con Repository, Factory Method, DTO y JWT).
- **Arquetipo Maven:** Base reutilizable para los microservicios.

Todos los componentes han sido versionados en GitHub siguiendo estrategia GitHub Flow y cumplen con los requerimientos de la Evaluación Parcial 2.

Patrones de Diseño Implementados:

Backend:

- **Repository Pattern:** En ProyectoRepository y RecursoRepository extendiendo JpaRepository.
- **Factory Method:** En ProyectoFactory y RecursoFactory para crear entidades con valores por defecto.
- **DTO Pattern:** Uso de Request y Response para separar entidades del dominio.

BFF:

- **Circuit Breaker:** Implementado con Resilience4j (@CircuitBreaker) y métodos de fallback.

Frontend:

- **Component-Based Architecture:** Componente ProjectCard.jsx como módulo NPM.
- **Observer Pattern:** Mediante useState + useEffect.

12. Estrategia de Branching

Para el control de versiones del proyecto se seleccionó la estrategia **GitHub Flow**.

Justificación de la elección

Se evaluaron tres estrategias principales antes de tomar la decisión:

- **Git Flow:** Utiliza varias ramas permanentes (main, develop, release, hotfix). Es muy estructurado y adecuado para proyectos grandes con ciclos de release formales, pero resultó **demasiado complejo** y pesado para el alcance y tiempo limitado de esta evaluación.
- **Trunk-Based Development:** Consiste en trabajar **directamente sobre la rama principal (main)**, sin ramas de feature largas. Aunque es muy eficiente en equipos maduros con excelente automatización de pruebas, requiere una

disciplina muy alta y feature flags, lo que no era práctico para este proyecto académico.

- **GitHub Flow:** Fue la estrategia elegida porque ofrece el **mejor equilibrio** entre simplicidad y control. Permite desarrollar funcionalidades en ramas cortas (feature/*), realizar revisiones mediante Pull Requests y mantener la rama main siempre estable y desplegable.

Esta elección se adaptó perfectamente al tamaño del equipo, al tiempo disponible y a las necesidades del proyecto, permitiendo un desarrollo ágil, organizado y con buena trazabilidad.

13. Evaluación general y Conclusiones

La implementación realizada en la **Evaluación Parcial N°2** ha permitido materializar completamente la arquitectura propuesta en el Parcial 1, cumpliendo con todos los requerimientos de la asignatura.

Se lograron los siguientes beneficios gracias al diseño basado en microservicios:

- Reducción del tiempo de respuesta
- Disminución del acoplamiento
- Aumento de la escalabilidad
- Mejora en la mantenibilidad

Se desarrollaron exitosamente:

- Un **componente Frontend** en React + Vite con módulo NPM reutilizable (ProjectCard)
- Un **Backend For Frontend (BFF)** con orquestación y Circuit Breaker
- Dos **microservicios** (Gestión de Proyectos y Gestión de Recursos) basados en el arquetipo Maven
- Patrones de diseño clave: Repository, Factory Method, DTO, Circuit Breaker y Component-Based Architecture
- Estrategia **GitHub Flow** para el control de versiones

La solución entregada es **escalable, resiliente, mantenible y segura** mediante JWT, cumpliendo el patrón Database-per-Service. El BFF mejora significativamente la experiencia del frontend al centralizar la lógica de negocio.

En conclusión, esta arquitectura moderna resuelve los problemas planteados por **Innovatech Solutions** (visibilidad en tiempo real, asignación eficiente de recursos y monitoreo de KPI), proporcionando una base sólida, desacoplada y preparada para entornos distribuidos y futuros escalamientos.

Referencias

Auth0. (2023). JSON Web Token Introduction. <https://auth0.com/learn/json-web-tokens/>

Fowler, M. (2018). Patterns of Enterprise Application Architecture. Addison-Wesley.

Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems. O'Reilly Media.

Richardson, C. (2019). Microservices Patterns. Manning Publications.

Villamizar, M., Garcés, O., Castro, H., Verano, M., Salamanca, L., Casallas, R., & Gil, S. (2015). Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud. 2015 10th Computing Colombian Conference (10CCC), 583-590. <https://doi.org/10.1109/ColumbianCC.2015.7333476>